Adaptive RDMA Optimization for Distributed LLM Training and Inference on Consumer Hardware

A Unified Communication Framework Addressing Fragmentation
Across NCCL, UCX, JACCL, and Emerging Standards

Nathan [Last Name]
MS Computer Science Candidate
SUNY New Paltz
[email]

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

Abstract

The distributed machine learning ecosystem suffers from fundamental architectural fragmentation across communication standards, creating compounding failures that block efficient training of large Mixture-of-Experts (MoE) models on consumer GPU clusters. This proposal presents a unified middleware framework that addresses seven critical blocking issues identified across NCCL, UCX, vLLM, Flash Attention, MLX/JACCL, and Exo: (1) cudaMallocAsync IPC incompatibility, (2) transport selection robustness failures, (3) container shared memory isolation, (4) cross-NUMA GPU latency regression, (5) RoCE reliability timeouts, (6) heterogeneous cluster build requirements, and (7) NCCL-UCX container integration. Our approach introduces a Weighted Finite Automaton (WFA) classifier for tensor-aware adaptive routing that operates above the transport layer, enabling intelligent differentiation of hot/warm/cold data paths without requiring modifications to existing RDMA stacks. We target 30–50% reduction in Time-To-First-Token (TTFT) latency for distributed inference workloads on commodity hardware configurations including RTX 3090/5090 clusters connected via 10GbE with iWARP/RoCE capabilities.

Contents

1 Introduction and Problem Statement **3**
1.1 The Multi-Standard Fragmentation Crisis 3
1.2 Research Objectives 3

2 Critical Issues Analysis **4**
2.1 Issue 1: cudaMallocAsync IPC Incompatibility (UCX #7110) 4

2.2 Issue 2: Transport Selection Robustness (UCX #9908) 5
2.3 Issue 3: Container Shared Memory Isolation (UCX #10055) 5
2.4 Issue 4: Cross-NUMA GPU Latency Regression (UCX #3249) 6
2.5 Issue 5: RoCE Reliability Timeouts (UCX #6000) 7
2.6 Issue 6: Heterogeneous Cluster Builds (UCX #10273) 7
2.7 Issue 7: NCCL-UCX Container Integration (UCX #10055) 8

3 Proposed Architecture **8**
3.1 System Overview 8
3.2 WFA Tensor Classifier Design 9
3.3 Data Flow Architecture 9

4 Implementation Plan **9**
4.1 Hardware Testbed 9
4.2 12-Month Timeline 9
4.3 Software Stack 9

5 Validation Strategy **12**
5.1 Simulation with UET-HTSIM 12
5.2 Performance Metrics 13
5.3 MoE Training Validation 13

6 Expected Contributions **13**

7 Conclusion **14**

1 Introduction and Problem Statement

1.1 The Multi-Standard Fragmentation Crisis

Modern distributed machine learning infrastructure operates across incompatible communication layers that were designed independently for different hardware ecosystems. As shown in Figure 1, applications must navigate at least five distinct communication paths depending on hardware configuration, with no unified abstraction that preserves performance while enabling portability.

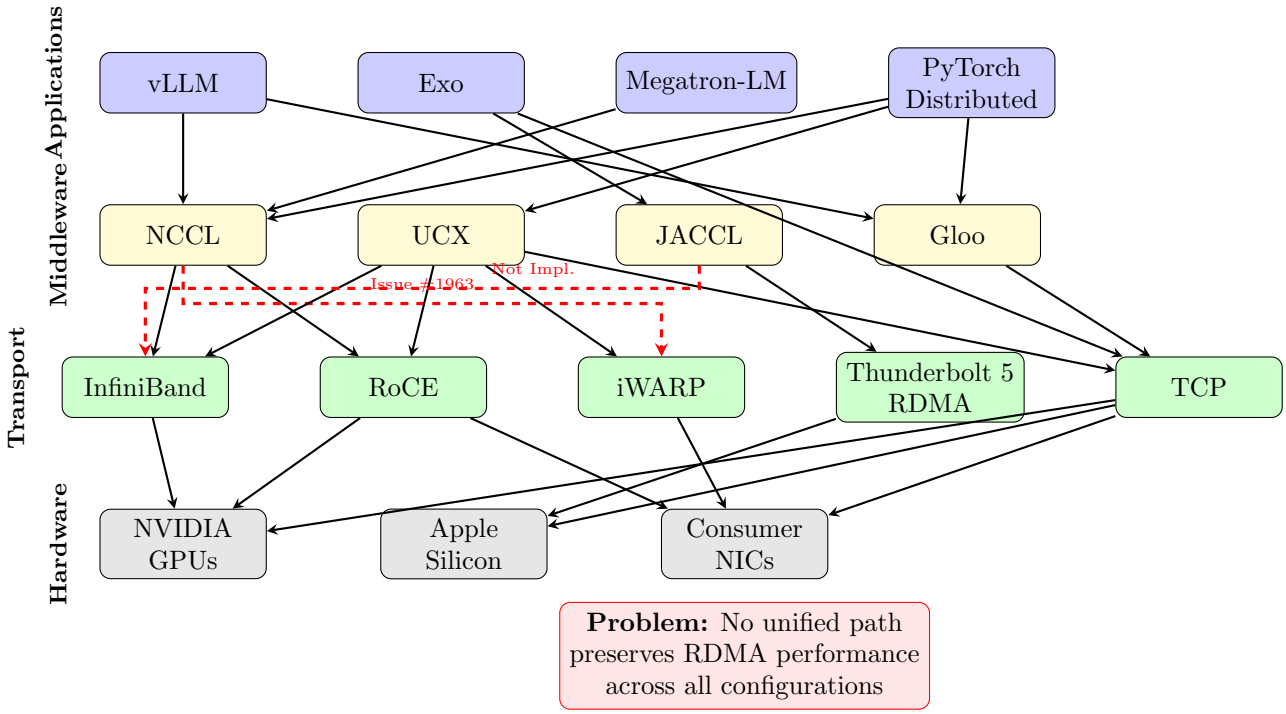

Figure 1: Current fragmentation in distributed ML communication stacks. Dashed red lines indicate broken or unimplemented paths identified through GitHub issue analysis.

1.2 Research Objectives

This thesis proposes to:

O1. Design and implement a unified middleware layer that abstracts transport-specific details while preserving RDMA performance characteristics

O2. Develop a Weighted Finite Automaton (WFA) classifier that categorizes tensor access patterns as hot/warm/cold for differential routing

O3. Address the seven critical blocking issues preventing UCX adoption as a unified communication standard

O4. Validate the approach through both simulation (UET-HTSIM) and real hardware deployment (RTX 3090/5090 cluster with 10GbE iWARP)

O5. Demonstrate applicability to MoE model training (Granite 4/Mamba 2 architectures) for RISC-V assembly generation

2 Critical Issues Analysis

This section details the seven blocking issues identified through comprehensive GitHub issue analysis across NCCL, UCX, vLLM, Flash Attention, MLX/JACCL, and Exo repositories.

2.1 Issue 1: cudaMallocAsync IPC Incompatibility (UCX #7110)

Challenge 1. Modern PyTorch memory pools use `cudaMallocAsync` for efficient GPU memory allocation, but these allocations do not support `cuIpcGetMemHandle`, breaking GPU RDMA for inter-process communication.

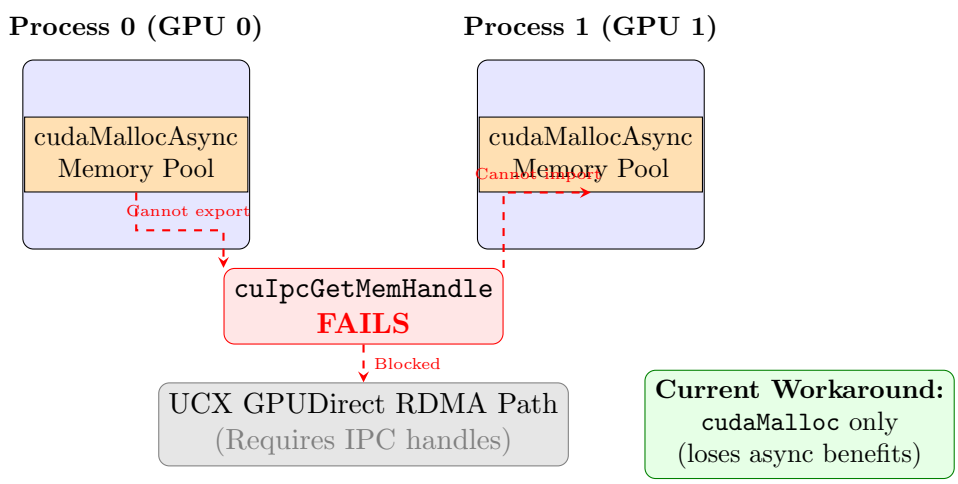

Figure 2: cudaMallocAsync IPC incompatibility blocks modern PyTorch memory pools from using UCX RDMA.

Proposed Solution 1. We propose a **Memory Pool Proxy Layer** that:

1. Intercepts tensor allocations before they reach PyTorch's memory allocator
2. Maintains a shadow pool of IPC-exportable memory regions pre-registered with UCX
3. Performs transparent copy-on-RDMA-request for async-allocated tensors that require inter-process transfer
4. Caches frequently-transferred tensors in the IPC-compatible pool based on WFA classifier predictions

2.2 Issue 2: Transport Selection Robustness (UCX #9908)

Challenge 2. UCX fails when `UCX_TLS` is set to any transport other than "rc" (Reliable Connection) for multi-node training, limiting transport flexibility and preventing automatic fallback.

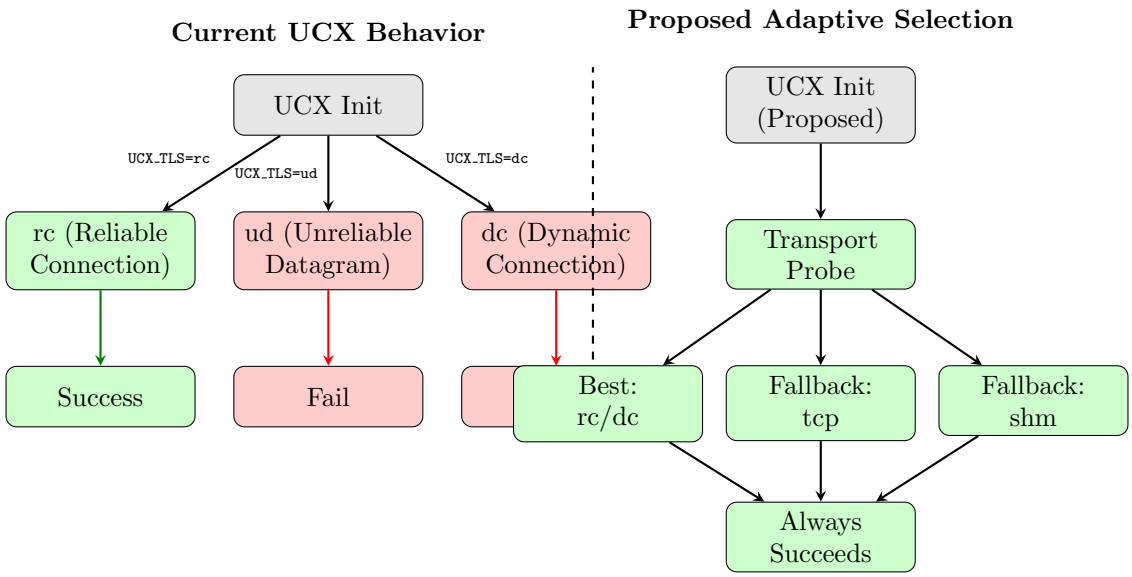

Figure 3: Transport selection failure modes in current UCX vs. proposed adaptive selection with graceful fallback.

Proposed Solution 2. Implement a **Transport Capability Negotiation Protocol** that:

1. Probes available transports at connection establishment time
2. Maintains a priority list: {dc, rc, ud, tcp, shm} with dynamic reordering based on observed performance
3. Automatically falls back to lower-priority transports on failure
4. Exposes transport selection decisions to the WFA classifier for learning

2.3 Issue 3: Container Shared Memory Isolation (UCX #10055)

Challenge 3. NCCL all-reduce operations fail with "Child failed to attach to shmseg" errors when running in containers on the same server, even with `--ipc=host` Docker flags.

Proposed Solution 3. Our middleware introduces a **Container-Aware Communication Broker**:

1. Detects container boundaries via cgroup inspection
2. Establishes a host-level coordinator process with access to unified `/dev/shm`

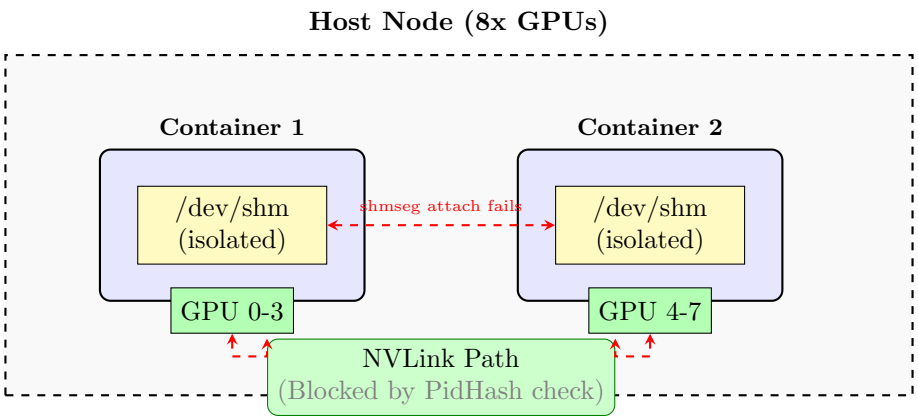

Figure 4: Container isolation prevents shared memory IPC between NCCL processes, even when GPUs are physically connected via NVLink.

3. Routes intra-node communication through the coordinator when direct shm attachment fails
4. Falls back to NVLink P2P with modified PidHash validation (requires `NCCL_HOSTID=0.0.0.0`)

2.4 Issue 4: Cross-NUMA GPU Latency Regression (UCX #3249)

Challenge 4. GPU-to-GPU communication experiences 10× latency degradation for messages larger than 262KB when GPUs reside in different NUMA domains.

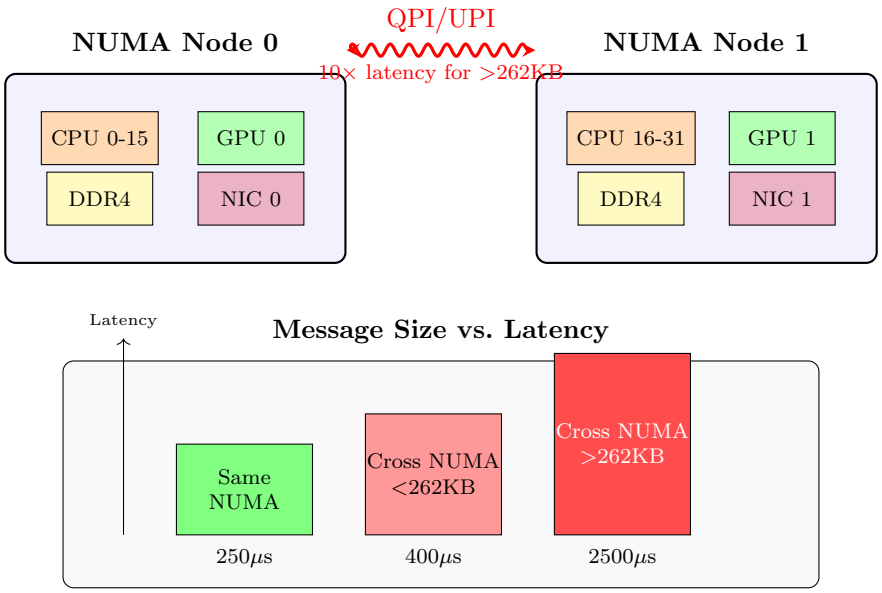

Figure 5: Cross-NUMA GPU communication latency explodes for larger message sizes due to memory controller contention.

Proposed Solution 4. The WFA classifier integrates **NUMA-Aware Tensor Placement**:

1. Queries sysfs topology to determine GPU-NIC-CPU NUMA relationships
2. Fragments messages >256KB crossing NUMA boundaries into pipelined chunks
3. Prioritizes NUMA-local bounce buffers for staging cross-domain transfers
4. Preemptively migrates frequently-accessed tensors to NUMA-optimal locations

2.5 Issue 5: RoCE Reliability Timeouts (UCX #6000)

Challenge 5. Multi-node RoCE deployments crash after approximately 90 seconds with "Transport retry count exceeded on mlx5/RoCE" errors, making long-running training jobs impossible.

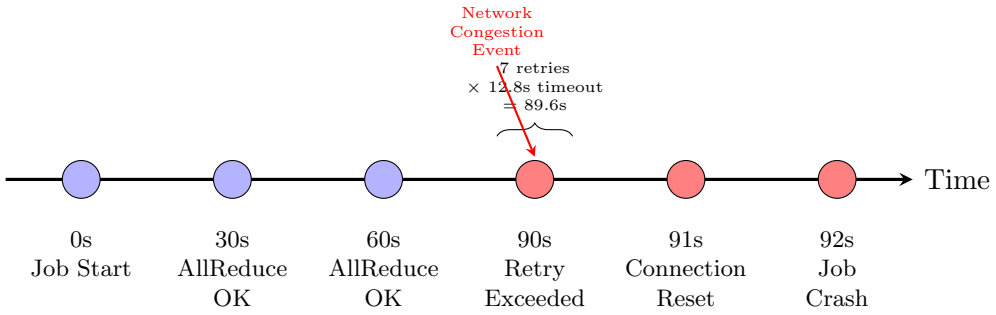

Figure 6: RoCE retry timeout accumulation leads to catastrophic failure after ∼90 seconds of congestion.

Proposed Solution 5. Implement **Proactive Congestion Detection and Path Switching**:

1. Monitor RTT variance and ECN marks to detect congestion before retry exhaustion
2. Maintain backup QP connections that can be hot-swapped on primary path degradation
3. Integrate with WFA classifier to preemptively route sensitive tensors away from congested paths
4. Implement application-level ACK coalescing to reduce retry pressure

2.6 Issue 6: Heterogeneous Cluster Builds (UCX #10273)

Challenge 6. UCX cannot be built as a single binary that works across both GPU-enabled and CPU-only nodes, requiring separate builds and complicating deployment.

Proposed Solution 6. Our middleware provides a **Capability Abstraction Layer**:

1. Wraps UCX initialization with runtime CUDA availability detection
2. Gracefully degrades GPU-specific features when CUDA libraries are unavailable

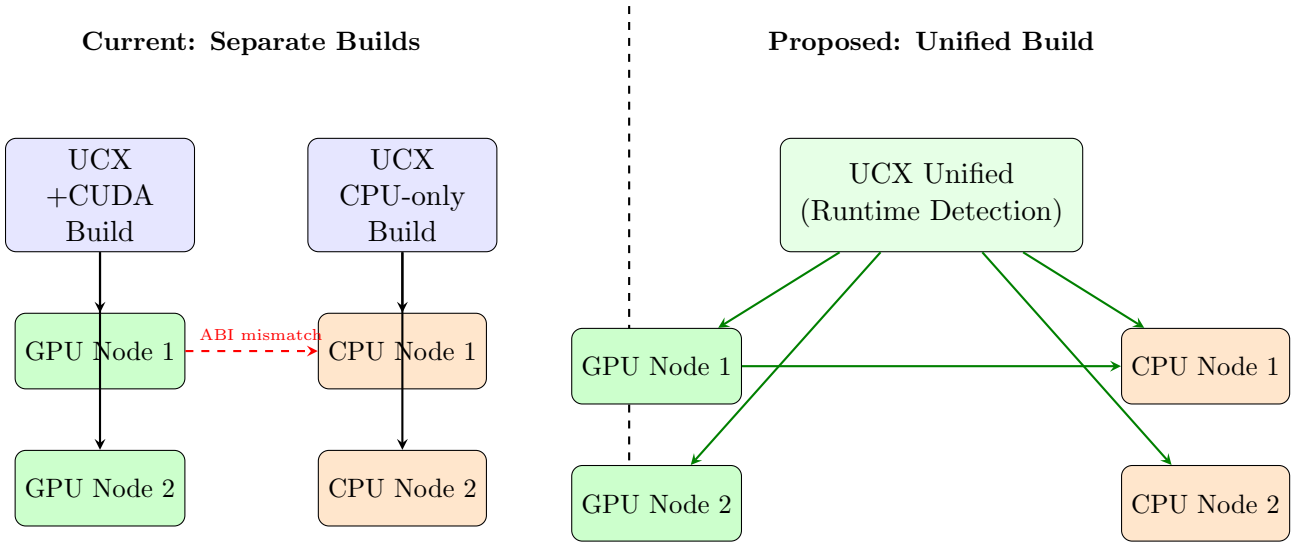

Figure 7: Heterogeneous cluster deployment requires separate UCX builds in current approach vs. proposed unified runtime-detection build.

3. Maintains ABI compatibility across node types through stable interface versioning
4. Enables mixed CPU/GPU deployments for MoE expert placement optimization

2.7 Issue 7: NCCL-UCX Container Integration (UCX #10055)

Challenge 7. Running NCCL with UCX as the network backend in containerized environments produces "TL_SHM ERROR" failures due to shared memory namespace isolation.

Proposed Solution 7. Automatic **Container Environment Detection and Transport Filtering**:

1. Detect containerization via `/proc/1/cgroup` inspection
2. Automatically exclude shm transport when running in containers
3. Provide environment variable presets for common container runtimes (Docker, Kubernetes, Singularity)
4. Integrate fix into middleware initialization to eliminate manual configuration

3 Proposed Architecture

3.1 System Overview

Figure 9 presents the complete system architecture, showing how the proposed middleware layer integrates with existing communication stacks.

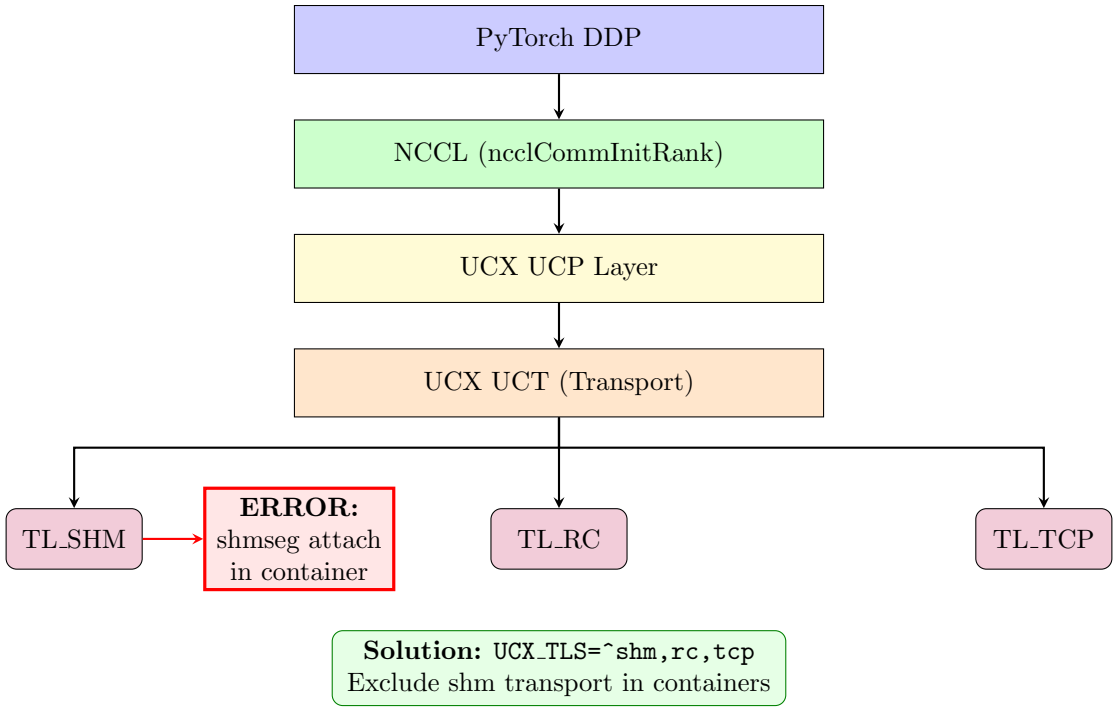

Figure 8: NCCL-over-UCX fails in containers when UCX attempts to use shared memory transport.

3.2 WFA Tensor Classifier Design

The Weighted Finite Automaton classifier operates on tensor access patterns to categorize data transfers into three classes:

- **Hot**: Frequently accessed, latency-critical (KV cache lookups, attention scores)
- **Warm**: Moderately accessed, throughput-sensitive (layer activations)
- **Cold**: Infrequently accessed, can tolerate delay (gradient accumulation buffers)

3.3 Data Flow Architecture

4 Implementation Plan

4.1 Hardware Testbed

4.2 12-Month Timeline

4.3 Software Stack

Listing 1: Core middleware interface design

```
class AdaptiveRDMAMiddleware:
```

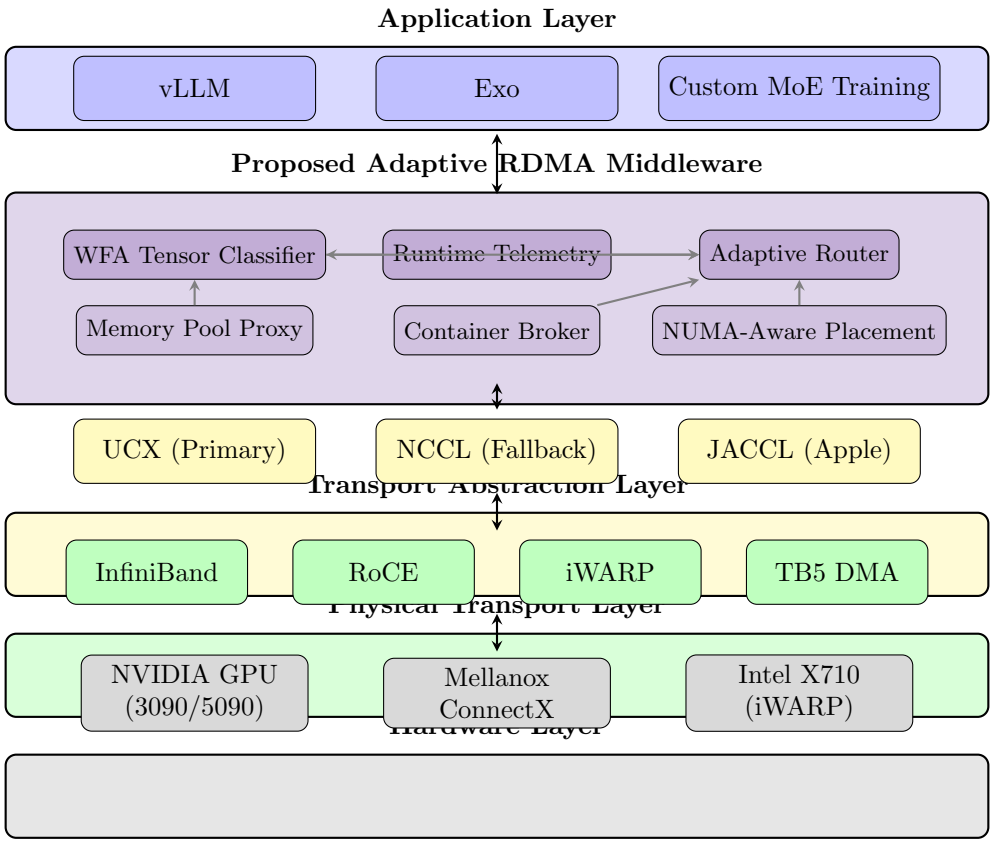

Figure 9: Complete system architecture showing the proposed adaptive RDMA middleware layer and its integration with existing communication stacks.

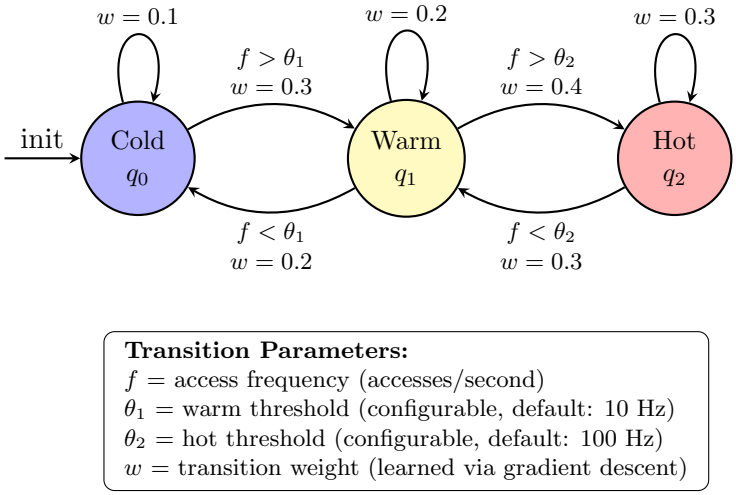

Figure 10: Weighted Finite Automaton for tensor classification. Transition weights are learned from runtime telemetry.

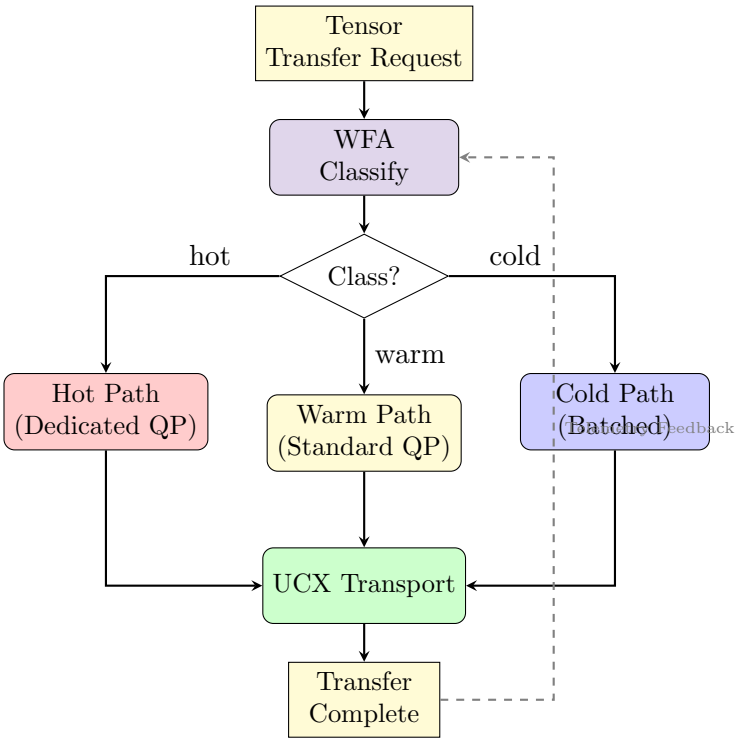

Figure 11: Data flow through the adaptive RDMA middleware showing classification and routing paths.

Table 1: Available hardware resources for implementation and validation

System	**GPUs**	**Network**	**Role**
Cerberus	2× RTX 5090 (64GB total)	10GbE (iWARP)	Primary inference node
Chimera	3× RTX 3090 (72GB total)	10GbE (iWARP)	Training/secondary inference
Lab NIC Pool	Mellanox ConnectX-4 IB	100GbE (upgrade path)	High-bandwidth experiments
Intel X710	–	10GbE iWARP	Consumer RDMA validation

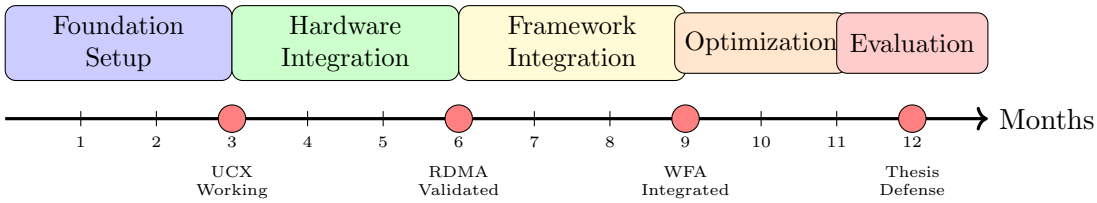

Figure 12: Implementation timeline with key milestones.

```python
    def __init__(self, config: MiddlewareConfig):
        self.wfa_classifier = WFAClassifier(config.wfa_params)
        self.telemetry = RuntimeTelemetry()
        self.transport = self._init_transport(config)

    def _init_transport(self, config):
        # Automatic transport selection with fallback
        if self._detect_container():
            config.ucx_tls = "^shm,rc,tcp"  # Issue #10055 fix

        try:
            return UCXTransport(config)
        except UCXInitError:
            return NCCLFallback(config)

    async def transfer_tensor(self, tensor: Tensor, dest: int):
        # WFA classification
        classification = self.wfa_classifier.classify(
            tensor.id, tensor.size, self.telemetry.get_access_pattern(
                tensor.id)
        )

        # Route based on classification
        if classification == TensorClass.HOT:
            return await self.transport.send_priority(tensor, dest)
        elif classification == TensorClass.WARM:
            return await self.transport.send_standard(tensor, dest)
        else:  # COLD
            return await self.transport.send_batched(tensor, dest)
```

5 Validation Strategy

5.1 Simulation with UET-HTSIM

Before hardware deployment, we will validate the WFA classifier and adaptive routing algorithms using the Ultra Ethernet Consortium's HTSIM simulator:

1. Model the 5-node cluster topology (2×5090 + 3×3090)
2. Generate traffic matrices matching LLM inference patterns:
    - Incast (all-reduce gradients)
    - KV cache scatter (point-to-multipoint)
    - Activation tensors (point-to-point)

3. Compare Flow Completion Time (FCT) distributions against baseline

5.2 Performance Metrics

Table 2: Target performance improvements

Metric	Baseline	Target
Time-To-First-Token (TTFT)	Current vLLM	30–50% reduction
Training iteration time	NCCL baseline	20–30% reduction
Cross-NUMA latency (¿256KB)	2500μs	¡500μs
RoCE session stability	90s before crash	¿24h continuous
Container deployment time	Manual config	Zero-config

5.3 MoE Training Validation

Final validation will demonstrate training of a Granite 4 / Mamba 2 style MoE model for RISC-V assembly generation:

1. Expert parallelism distributed across Cerberus + Chimera
2. AllToAll communication for expert routing
3. Measure expert utilization and communication overhead
4. Compare training throughput with and without adaptive middleware

6 Expected Contributions

1. **Unified Middleware Framework**: First open-source middleware that enables UCX-based RDMA across NVIDIA, Apple Silicon, and consumer hardware with automatic transport selection and fallback.
2. **WFA Tensor Classifier**: Novel application of weighted finite automata for runtime classification of ML tensor access patterns, enabling intelligent routing decisions.
3. **Issue Resolution**: Practical solutions to seven critical GitHub issues blocking distributed ML infrastructure adoption.
4. **Performance Validation**: Comprehensive benchmarks comparing adaptive RDMA optimization against baseline NCCL/UCX on commodity hardware.
5. **Open-Source Release**: All code, configurations, and documentation released under permissive license for community adoption.

7 Conclusion

The distributed ML ecosystem's fragmentation across communication standards creates compounding failures that block efficient training of large models on consumer hardware. This proposal presents a systematic approach to addressing these challenges through an adaptive middleware layer that:

- Abstracts transport-specific details while preserving RDMA performance
- Intelligently classifies tensor access patterns for differential routing
- Automatically handles container isolation, NUMA topology, and heterogeneous deployments
- Enables unified deployment across NVIDIA, Apple Silicon, and consumer NIC configurations

By addressing the seven critical blocking issues identified through comprehensive GitHub analysis, this work will enable researchers and practitioners to leverage RDMA benefits on commodity hardware without the current fragmentation penalty.

References

[1] Shamis, P., et al. "UCX: An Open Source Framework for HPC Network APIs and Beyond." *IEEE HOTI*, 2015.

[2] Jeaugey, S. "NCCL 2.0." *GPU Technology Conference*, 2017.

[3] Handley, M., et al. "Re-architecting Datacenter Networks and Stacks for Low Latency and High Performance." *ACM SIGCOMM*, 2017.

[4] Olteanu, A., et al. "EQDS: An Equivalent Queueing Discipline Scheduler." *USENIX NSDI*, 2022.

[5] Kwon, W., et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *ACM SOSP*, 2023.

[6] Ultra Ethernet Consortium. "UEC 1.0 Specification." 2025.

[7] Dao, T., et al. "FlashAttention: Fast and Memory-Efficient Exact Attention." *NeurIPS*, 2022.